1. Introduction to the System

The system is an **E-Commerce Order Management System (EOMS)** implemented in Java. It supports customer order placement, payment processing, inventory handling, shipping, reporting, and administrative interfaces. The architecture is designed to clearly define how the system's main components interact, enforce separation of responsibilities, and provide a stable foundation for future extension and evolution.

This report covers:

- the overall architecture of the system,
- the architectural patterns selected,
- the reasons behind those choices,
- the non-functional requirements influenced by these architectural decisions.

---

2. Architectural Design Decisions and Justification

2.1 Overall Architecture

The system is designed using a **Layered Architecture** combined with **Model-View-Controller (MVC)** organization.

Layered Architecture

The architecture is divided into distinct layers:

- **Presentation Layer**
  - Package: `com.eoms.Boundary`
  - Classes: `CheckoutView`, `PaymentView`, `OrderTrackingView`, `ProductCatalogView`, `AdminReportView`
- **Application / Controller Layer**
  - Package: `com.eoms.app`
  - Classes: `CustomerRoleHandler`, `AdminRoleHandler`, `EomsApplication`, `RoleHandler`
  - Providers/factories: `ProductTypeFactoryProvider`, `DefaultPaymentProcessorProvider`, `ReportStrategyProvider`
  - Mediators: `com.eoms.app.mediator.*`
- **Business / Service Layer**
  - Package: `com.eoms.service`
  - Classes: `OrderService`, `PaymentService`, `ProductService`, `ReportService`, `ShippingService`
  - Implementations: `com.eoms.service.impl`
- **Data Access Layer**
  - Package: `com.eoms.DAO`
  - Classes/interfaces: `OrderDAO`, `ProductDAO`, `PaymentDAO`, `ShipmentDAO`, `OrderInterface`, `ProductInterface`, `PaymentInterface`, `ShipmentInterface`
  - Iterators: `OrderIterator`, `OrderListIterator`
- **Infrastructure Layer**
  - Packages: `com.eoms.config`, `com.eoms.adapter`, `com.eoms.externalservice`
  - Components: `Logger`, `UIFactoryRegistry`, `PaymentGatewayAdapter`, `DHLShippingAdapter`

This structure enforces separation of responsibilities by isolating user interface, application coordination, business logic, data access, and external integrations into independent layers, reducing coupling and improving clarity.

All dependencies follow a unidirectional flow from outer layers to inner layers (Presentation → Application → Business → Data → Infrastructure). This ensures that high-level components do not depend on low-level implementation details and prevents tight coupling between modules.

Model-View-Controller (MVC)

MVC is applied primarily in the Presentation and Application layers:

- **Model**
  - Domain entities in `com.eoms.entity` such as `Order`, `Product`, `Payment`, `Shipment`
- **View**
  - Boundary classes in `com.eoms.Boundary` responsible for user interaction and display
- **Controller**
  - Handler classes in `com.eoms.app` such as `CustomerRoleHandler` and `AdminRoleHandler` that receive user input and delegate processing to the service layer

Controllers act as an indirection layer between the view and business logic, reducing direct coupling between user interface components and domain operations and improving system flexibility.

Within the application layer, mediator components coordinate interactions between services, centralizing workflow logic and preventing direct service-to-service dependencies, which further reduces coupling.

2.2 Why these patterns are suitable

Layered Architecture

- The system naturally separates into user interface, application coordination, business logic, persistence, and infrastructure concerns.
- It enforces clear responsibility boundaries, improving modularity and reducing interdependencies between components.
- It supports independent evolution of layers, allowing changes in UI, business rules, or data access without affecting the entire system.

MVC

- The system contains distinct user interaction components and domain logic, making MVC a natural fit.
- It separates user interface handling from business processing and domain data.
- It allows boundary views to evolve independently from controllers and services, improving flexibility and maintainability.

2.3 How the patterns support system requirements

Support from Layered Architecture

- **Separation of concerns**: Each layer is responsible for a specific aspect of the system.
- **Maintainability**: Changes are localized within layers and do not propagate across the system.
- **Clarity**: System flow is clearly structured from presentation to persistence.

Support from MVC

- **UI flexibility**: Views handle interaction while controllers manage request flow.
- **Testability**: Business logic and models can be tested independently of the UI.
- **Encapsulation**: Domain logic is isolated from presentation components.

2.4 How the patterns complement each other

- The **Layered Architecture** provides the vertical structure of the system.
- **MVC** provides horizontal separation within the upper layers:
  - The Presentation Layer is structured into View and Controller roles
  - The Business Layer handles model operations and domain logic

Together they ensure:

- UI changes do not affect business or data layers
- Business rules remain independent of presentation logic
- Data access is encapsulated and reusable

---

3. Identified Non-Functional Requirements

3.1 Performance

- The service layer centralizes business operations, avoiding redundant processing logic.
- The DAO layer isolates data access and iteration mechanisms, improving efficiency in data retrieval.
- The architecture reduces unnecessary communication between layers, improving overall system responsiveness.

3.2 Security

- Sensitive operations such as payment processing are handled within the service layer rather than the presentation layer.
- Layer isolation enforces controlled access, preventing presentation components from directly manipulating domain or persistence logic and reducing the system's attack surface.
- Infrastructure adapters encapsulate external integrations, protecting internal system components from exposure.

3.3 Availability

- Layer separation reduces the impact of failures, preventing issues in one layer from propagating across the entire system.
- Application controllers and services manage workflows and can handle partial failures before committing system state.
- Data access isolation allows persistence-related issues to be handled without affecting user interaction layers.

3.4 Maintainability

- Clear modular structure enables easier navigation and modification of system components.
- Changes are localized within specific layers, supporting long-term system evolution without widespread impact.
- New features (e.g., views, reports, services) can be added without modifying existing core logic.

3.5 Scalability

- The layered structure allows each part of the system to scale independently.
- New functionality can be added by extending existing layers rather than modifying core components.
- MVC separation enables independent evolution of user interfaces and backend logic.

---

4. Conceptual View (Block Diagram)

The conceptual view presents the system at a high level and focuses on the main subsystems and their interactions without exposing implementation details.

The E-Commerce Order Management System is organized into several major subsystems:

- Presentation subsystem responsible for user interaction
- Application subsystem responsible for request coordination and workflow management
- Service subsystem responsible for business logic execution
- Data access subsystem responsible for persistence operations

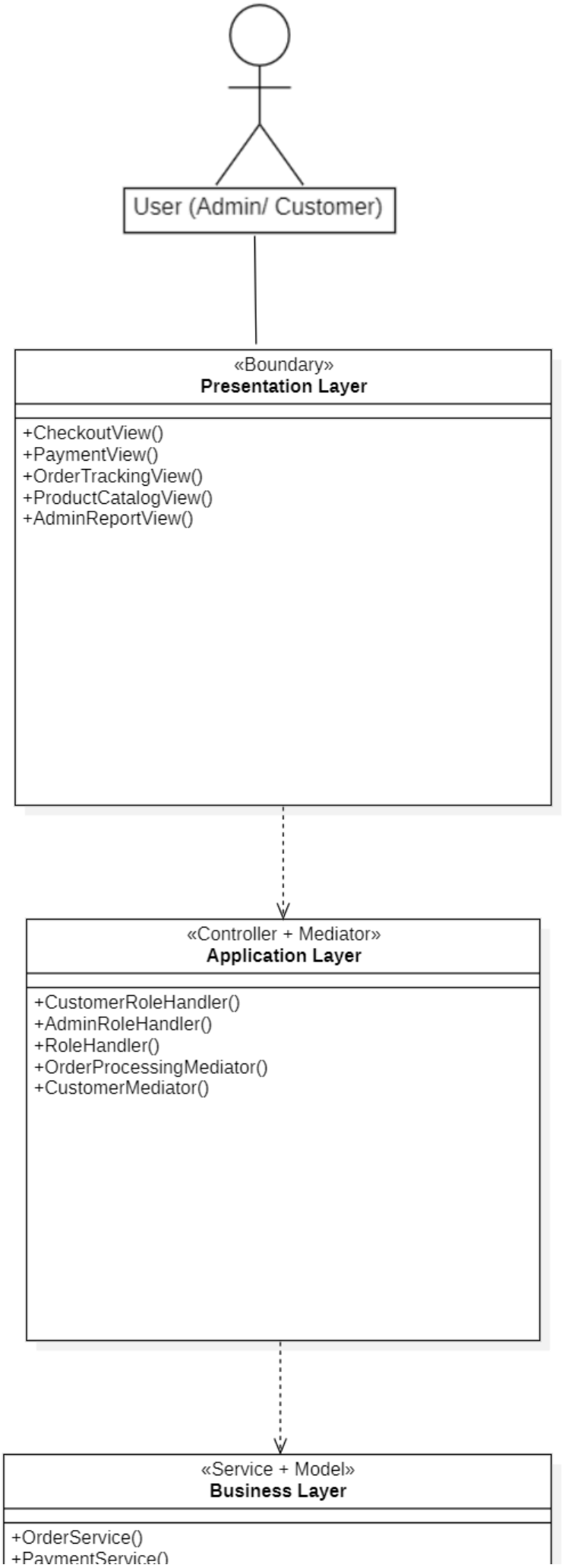
- External integration subsystem responsible for payment and shipping integrations

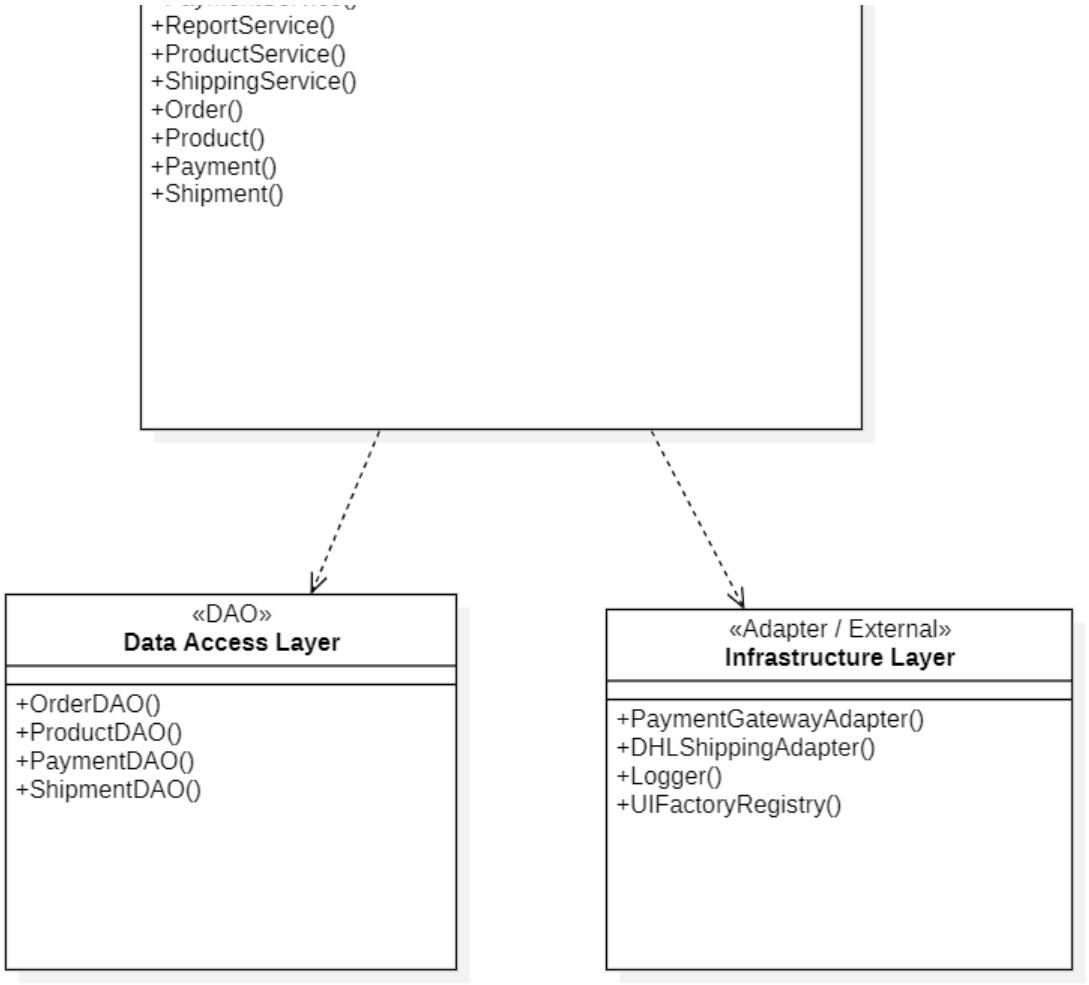
The conceptual organization of the system follows both Layered Architecture and MVC principles.

Main Interactions

- Customers and administrators interact with the Presentation Layer.
- Controllers coordinate requests and delegate operations to services.
- Services execute business rules and communicate with DAO components.
- DAO components manage persistence and retrieval operations.
- Infrastructure adapters communicate with external systems such as payment gateways and shipping providers.

Conceptual View Diagram


User (Admin/ Customer)
«Boundary»
Presentation Layer
+CheckoutView()
+PaymentView()
+OrderTrackingView()
+ProductCatalogView()
+AdminReportView()
«Controller + Mediator»
Application Layer
+CustomerRoleHandler()
+AdminRoleHandler()
+RoleHandler()
+OrderProcessingMediator()
+CustomerMediator()
«Service + Model»
Business Layer
+OrderService()



Explanation of the Diagram

The diagram shows the main system subsystems and their interactions.

---

5. Process View (Dynamic Behavior)

The process view illustrates the dynamic runtime behavior of the system during execution.

To represent the runtime interactions of the system, the "Place Order" scenario was selected because it involves the major architectural components and demonstrates the collaboration between views, controllers, services, DAOs, and external integrations.

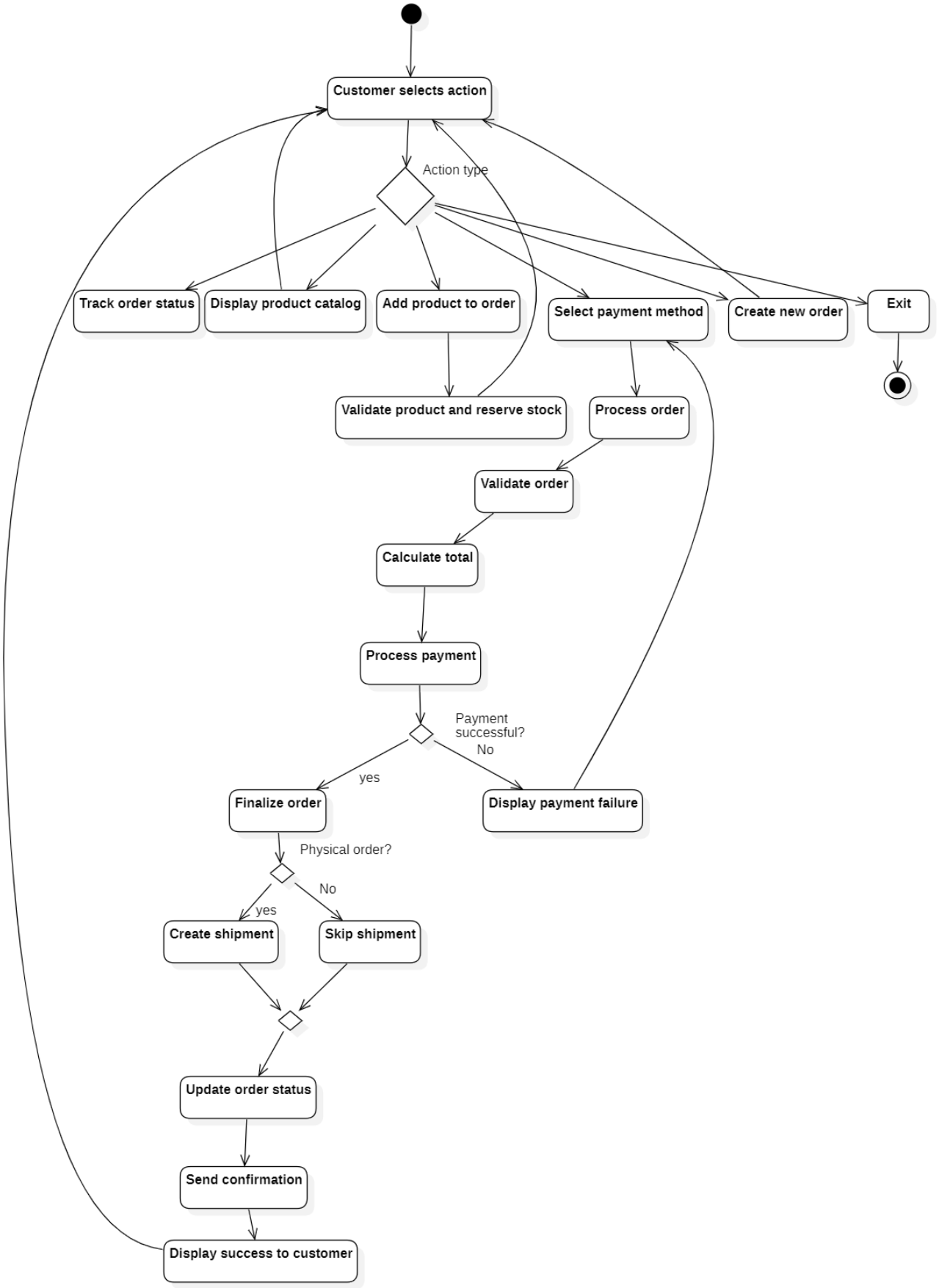
5.1 Activity Diagram

The activity diagram illustrates the workflow followed when a customer places an order.

Main Workflow

1. Customer browses products.
2. Customer selects products and proceeds to checkout.
3. System validates order information.
4. Payment processing is initiated.
5. Payment gateway validates transaction.
6. Order is created and stored.
7. Shipment is generated.
8. Order confirmation is returned to the customer.

Activity Diagram



Explanation of the Activity Diagram

The diagram illustrates the workflow of placing an order.

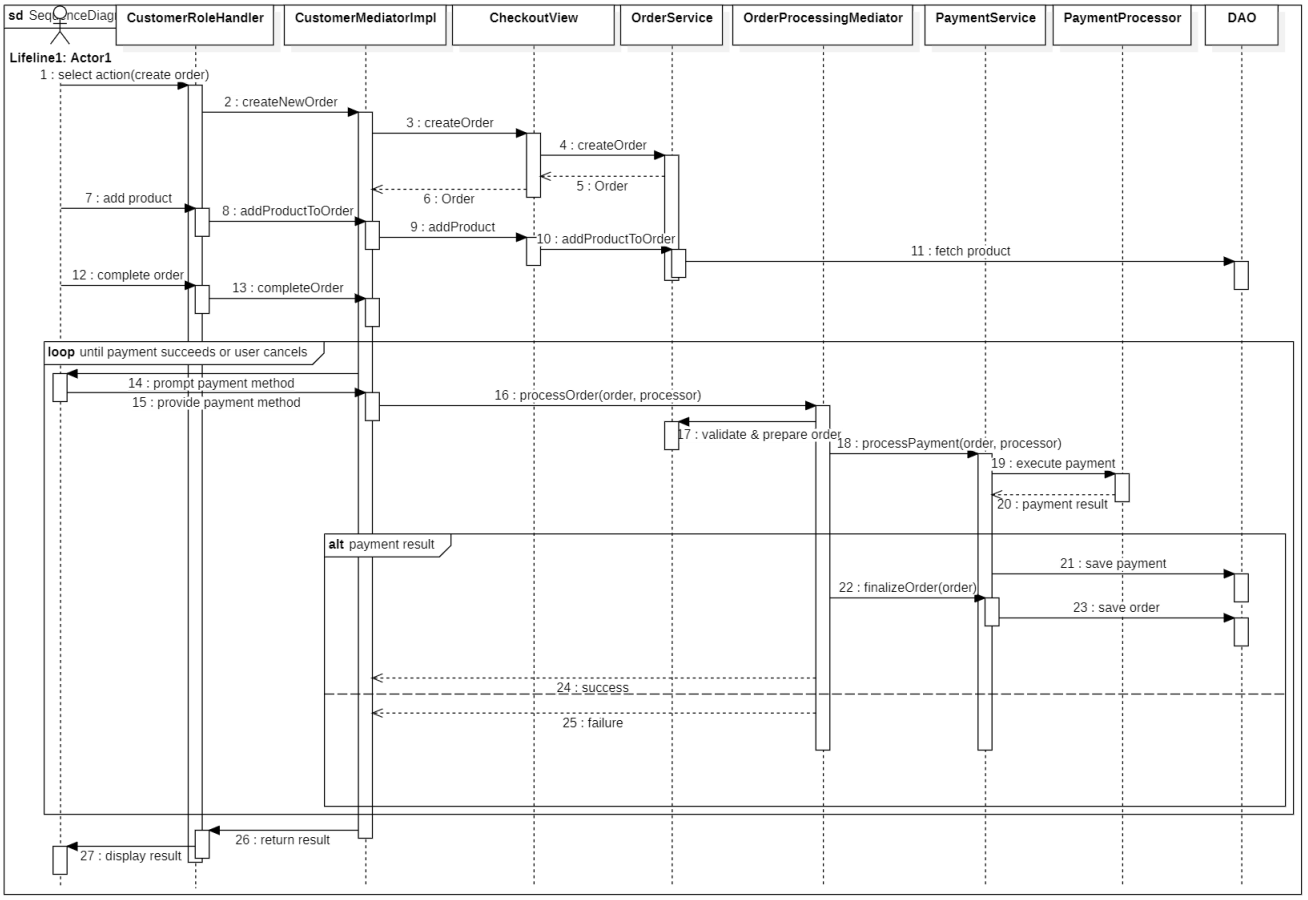
5.2 Sequence Diagram

The sequence diagram illustrates the runtime communication between system components during order placement.

Participating Components

- Customer
- CheckoutView
- CustomerRoleHandler
- OrderService
- PaymentService
- PaymentGatewayAdapter
- OrderDAO
- ShippingService
- ShipmentDAO

Sequence Diagram



Explanation of the Sequence Diagram

The diagram shows the runtime interaction between system components during order placement.

---

6. Development View (Component Diagram)

The development view describes how the software system is decomposed into modular components and how these components interact through interfaces and dependencies.

Because the system integrates both Layered Architecture and MVC, two component diagrams are used to clearly represent the architectural organization.

6.1 MVC Component Diagram

The MVC component diagram focuses on the interaction between views, controllers, and domain services.

Main MVC Components

View Components

- ProductCatalogView
- CheckoutView
- PaymentView
- OrderTrackingView
- AdminReportView

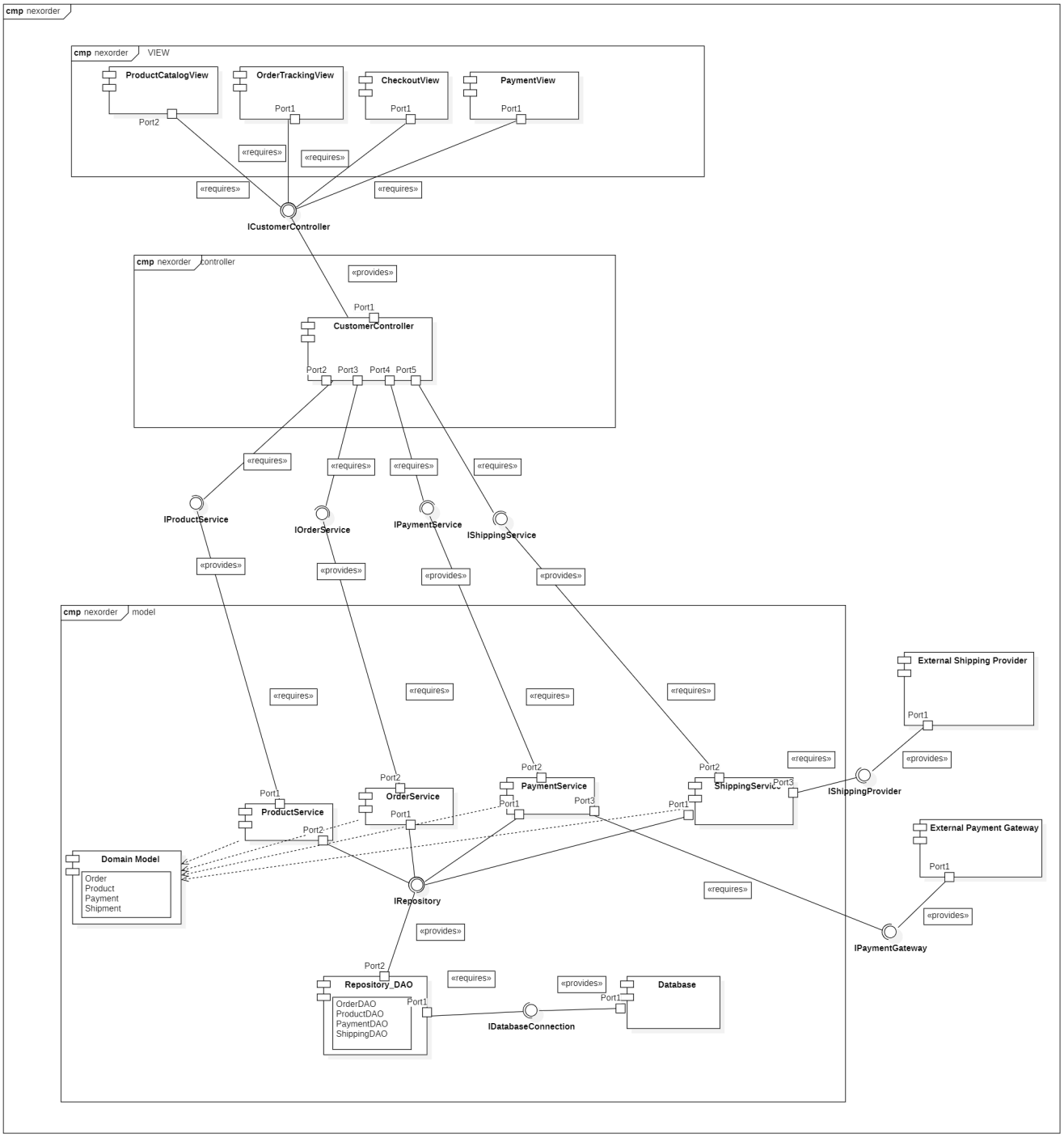
Controller Components

- CustomerRoleHandler
- AdminRoleHandler
- CustomerMediator

Model / Business Components

- OrderService
- ProductService
- PaymentService
- ShippingService
- ReportService

MVC Component Diagram



Explanation of the MVC Component Diagram

The diagram shows the separation between views, controllers, and services.

6.2 Layered Architecture Component Diagram

The layered component diagram focuses on the decomposition of the system into architectural layers.

Main Layers

Presentation Layer

Contains all boundary and interaction components.

Application Layer

Contains controllers, mediators, and workflow coordination logic.

Business Layer

Contains all business services and domain processing logic.

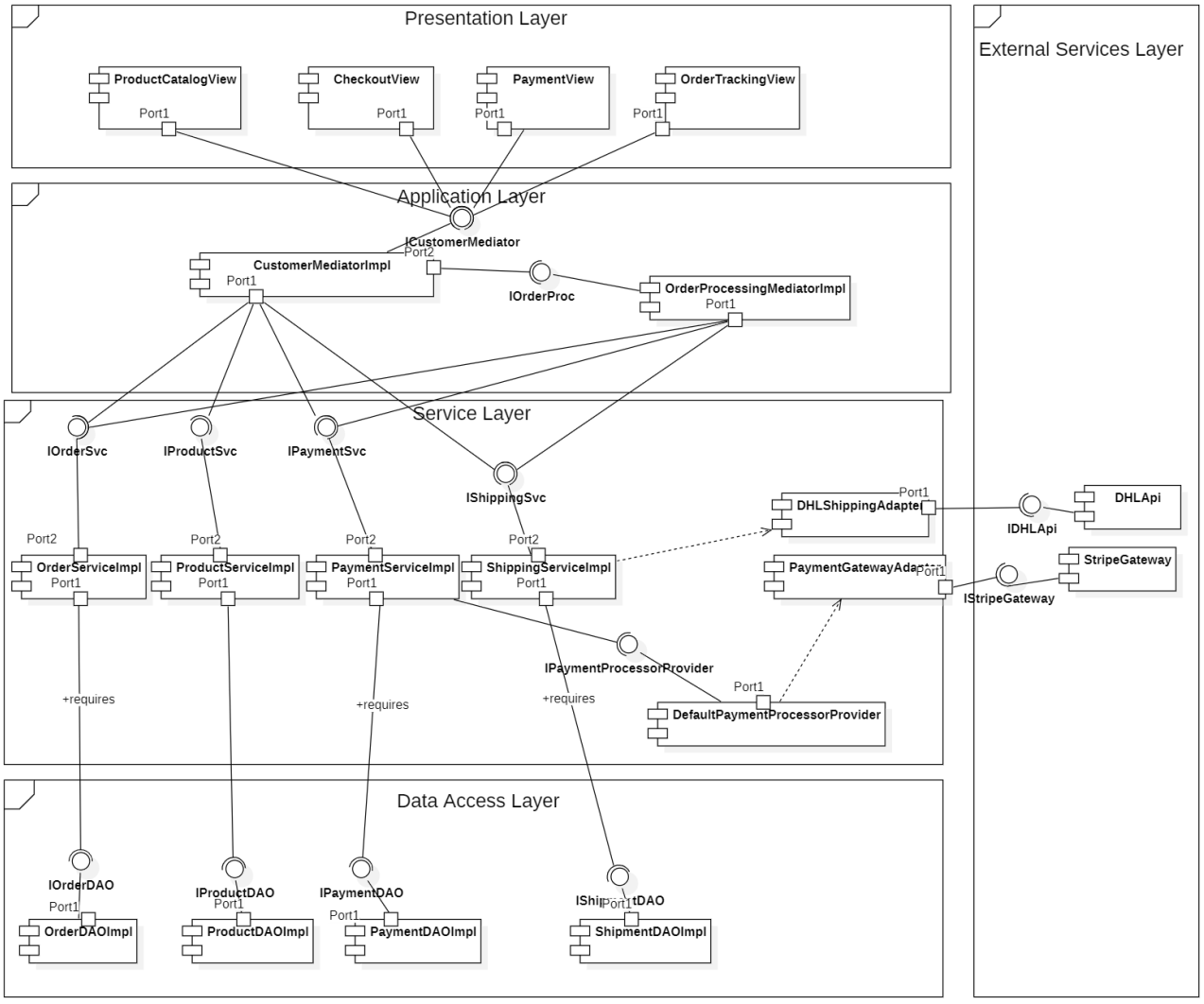
Data Access Layer

Contains DAO interfaces and persistence implementations.

External Services Layer

Contains external integrations.

Layered Component Diagram



Explanation of the Layered Component Diagram

The diagram illustrates the dependency structure between system layers.

---

7. Physical View (Deployment Diagram)

The physical view describes how software artifacts and components are deployed across hardware nodes and runtime environments.

Deployment Nodes

Client Device

- Runs presentation components and user interaction views.

- Allows customers and administrators to interact with the system.

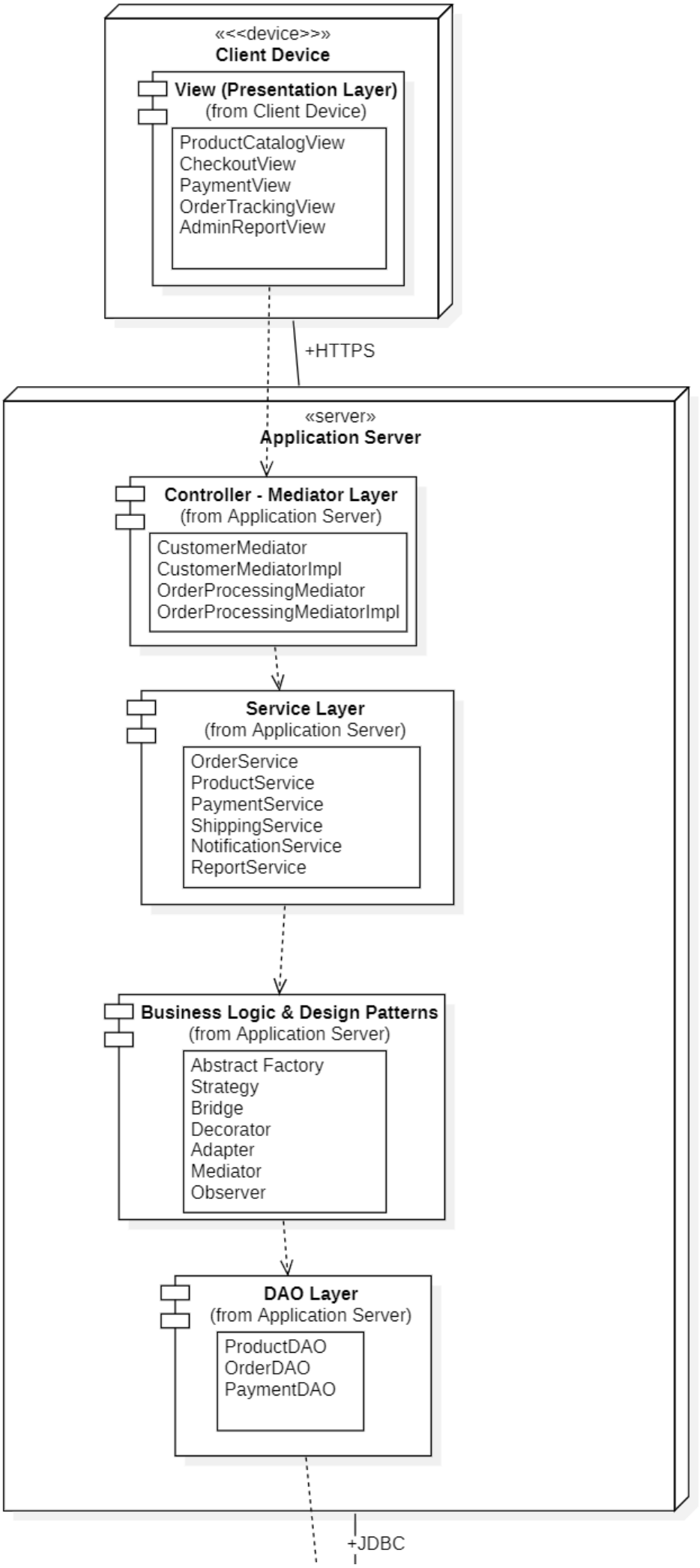
Application Server

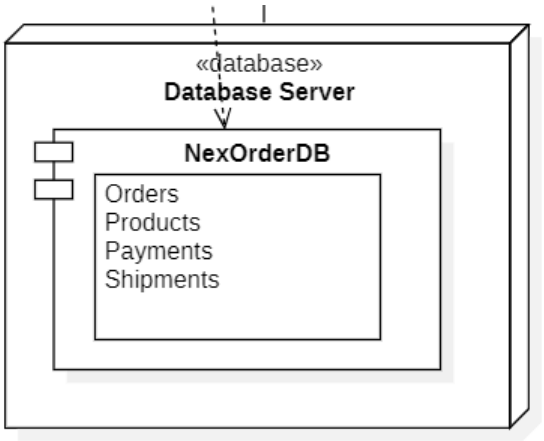
- Hosts application controllers, mediators, and business services.
- Coordinates workflows and executes business logic.

Database Server

- Stores persistent business data.
- Manages orders, products, payments, and shipment records.

Deployment Diagram


«<<device>>»
Client Device
View (Presentation Layer)
(from Client Device)
ProductCatalogView
CheckoutView
PaymentView
OrderTrackingView
AdminReportView
+HTTPS
«server»
Application Server
Controller - Mediator Layer
(from Application Server)
CustomerMediator
CustomerMediatorImpl
OrderProcessingMediator
OrderProcessingMediatorImpl
Service Layer
(from Application Server)
OrderService
ProductService
PaymentService
ShippingService
NotificationService
ReportService
Business Logic & Design Patterns
(from Application Server)
Abstract Factory
Strategy
Bridge
Decorator
Adapter
Mediator
Observer
DAO Layer
(from Application Server)
ProductDAO
OrderDAO
PaymentDAO
+JDBC



Explanation of the Deployment Diagram

The diagram shows the deployment of software components across hardware nodes.

---

8. Bonus Section — Reference Architecture

The E-Commerce Order Management System can be classified as a Web-Based Information System and a Transaction Processing System.

Relevance of the Reference Architecture

The system processes customer requests, manages persistent business data, handles payments, and executes order transactions.

These characteristics align with transaction processing architectures where users submit requests that are validated, processed, and committed to a database.

The system also follows the characteristics of web-based information systems because it:

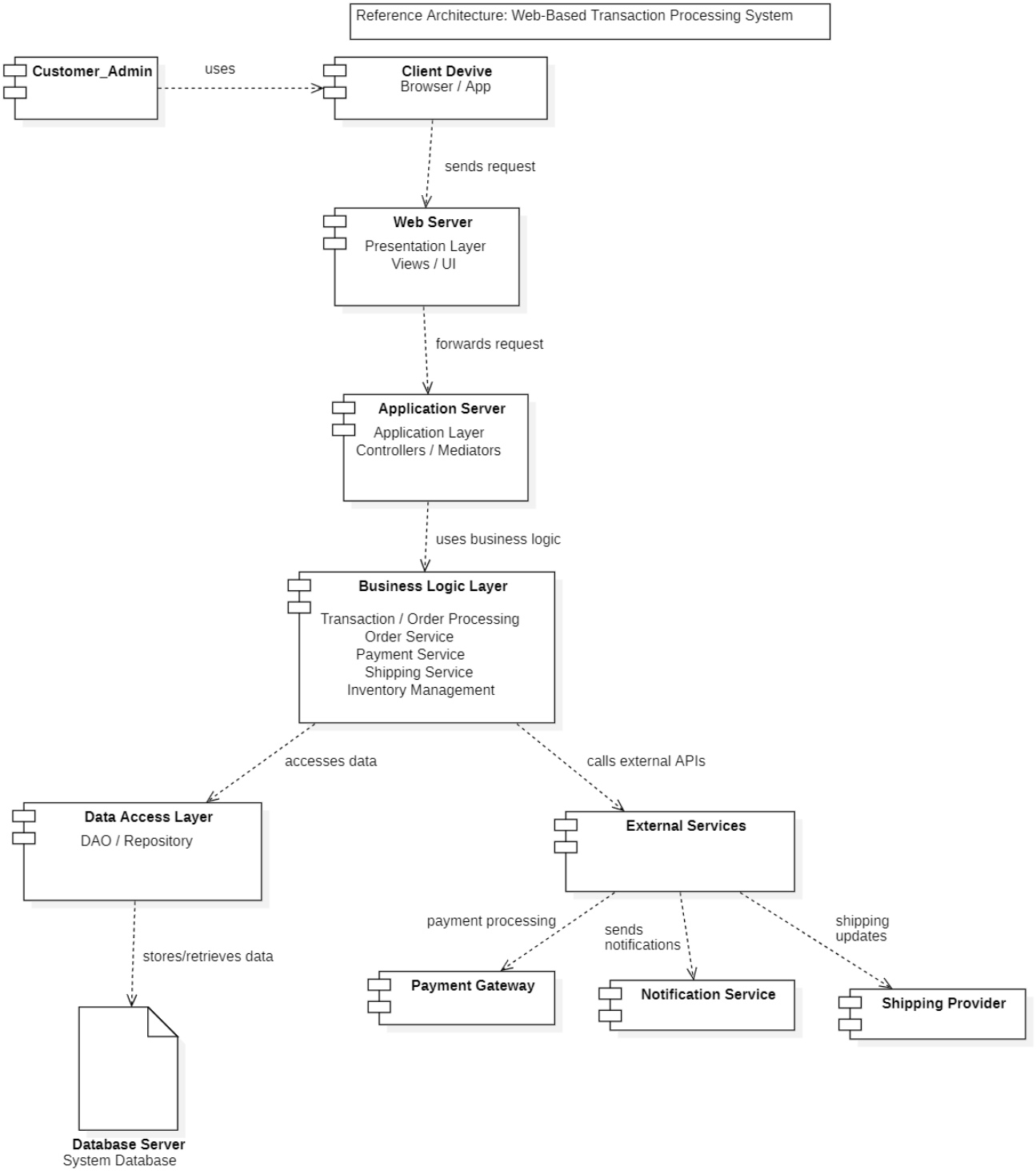
- provides user interaction interfaces,
- processes business operations,
- communicates with external services,
- and manages centralized information storage.

Mapping the System to the Reference Architecture

The system maps to the transaction processing architecture as follows:

- User Interface Layer → Presentation Views
- Transaction Coordination → Controllers and Mediators
- Business Processing → Service Layer
- Data Management → DAO and Database Layer
- External Integrations → Payment and Shipping Adapters

Reference Architecture Diagram



Explanation of the Reference Architecture Diagram

The diagram shows how the system maps to a transaction processing architecture.

---

9. Conclusion

The E-Commerce Order Management System was architected using a combination of Layered Architecture and Model-View-Controller organization.

These architectural styles provide clear separation of concerns, modularity, maintainability, and extensibility while supporting the system’s functional and non-functional requirements.

The conceptual, process, development, and physical views collectively document the structure and runtime behavior of the system from multiple perspectives.

The architecture also aligns with the characteristics of transaction processing and web-based information systems, making it suitable for scalable and maintainable e-commerce operations.

Github (Java code): https://github.com/Kiro-create/NexOrder